

**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript (parte 2)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

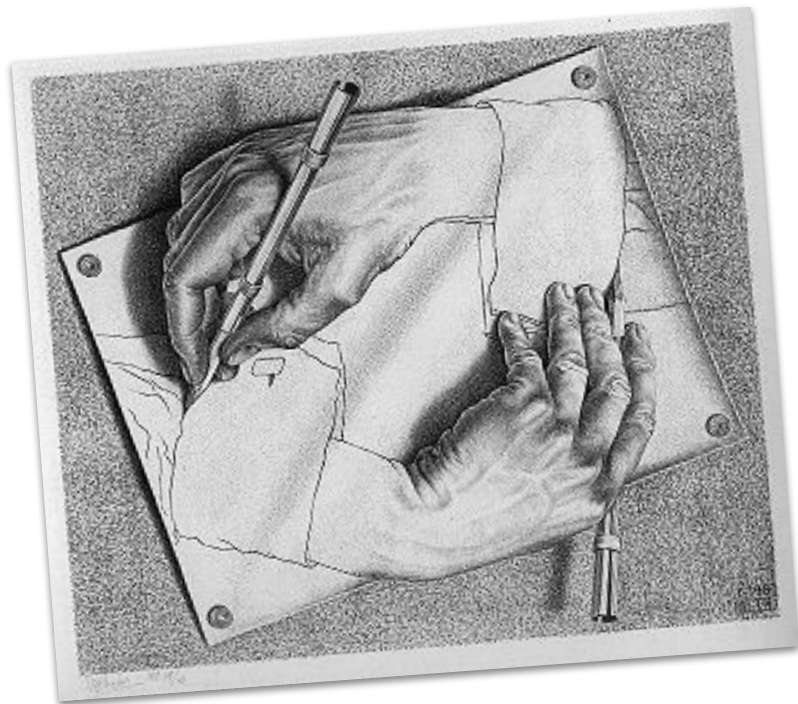
Roboto Mono



Recursão

o que é

- função que invoca a si mesma repetidamente até atingir uma condição de parada
- pode ser dividida em duas partes
 - caso base
 - passo indutivo
- equivalente à indução matemática



Recursão

vantagens

- simplificação de problemas complexos
- clareza e elegância

desvantagens

- custo de desempenho
- dificuldade de entendimento
- comparação com loops

exemplos

```
// função iterativa
let fatorial = numero => {
  let resultado = 1;

  for (let i = 1; i <= numero; i++) {
    resultado *= i;
  }

  return resultado;
}

// função recursiva
function fatorial (n) {

  if (n === 0 || n === 1) return 1;
  return n * fatorial(n - 1);
}
```



Recursão

vantagens

- simplificação de problemas complexos
- clareza e elegância

desvantagens

- custo de desempenho
- dificuldade de entendimento
- comparação com loops

exemplos

```
// função recursiva para imprimir todos os elementos html
function percorrerDOM(elemento) {

    if (elemento.tagName) {
        console.log(elemento.tagName);
    }

    let filhos = elemento.children;

    for (let i = 0; i < filhos.length; i++) {
        percorrerDOM(filhos[i]);
    }
}
```

Composição de Funções

o que é

- funções complexas a partir de funções simples
- o resultado de uma função é passado como entrada para outra



Composição de Funções

vantagens

- cada função realiza uma única operação (modularidade)
- funções pequenas pode ser reaplicadas em outras situações
- mais fáceis de manter e modificar

desvantagens

- dificuldade de depuração
- baixa legibilidade
- dificuldade com efeitos colaterais

exemplos

```
// composição manual
function dobrar(x) {
  return x * 2;
}

function somar(x) {
  return x + 1;
}

let resultado = somar(dobrar(5));

// composição por encadeamento de funções
function somaParesAoQuadrado(array) {
  return array
    .filter(numero => numero % 2 === 0)
    .map(numero => numero ** 2)
    .reduce((acc, numero) => acc + numero);
}
```



Composição de Funções

vantagens

- cada função realiza uma única operação (modularidade)
- funções pequenas pode ser reaplicadas em outras situações
- mais fáceis de manter e modificar

desvantagens

- dificuldade de depuração
- baixa legibilidade
- dificuldade com efeitos colaterais

exemplos

```
// pipe clássico
const ePar = n => n % 2 === 0;
const quadrado = n => n ** 2;
const soma = (a, b) => a + b;

const pipe = (...funcoes) => {
  return x => funcoes.reduce((v, f) => f(v), x);
};

const somaParesAoQuadrado = pipe(
  arr => arr.filter(ePar),
  arr => arr.map(quadrado),
  arr => arr.reduce(soma, 0)
);

somaParesAoQuadrado([1,2,3,4,5,6]); // 56
```



Closures

o que são

- conjunto composto pelo objeto função e seu escopo
- a função "lembra" do ambiente onde foi criada
- ocorre quando funções são criadas dentro de outras funções
- a função interna usa variáveis da função externa.



Closures

vantagens

- protegem dados e detalhes da implementação (encapsulamento)
- mantém o estado sem depender de variáveis globais ou objetos externos

desvantagens

- dificuldade de depuração
- uso excessivo de memória
- comportamento inesperado em loops

exemplos

```
function fatorialMemo() {  
  let cache = {};  
  
  return function fatorial(n) {  
    if (n in cache) return cache[n];  
  
    if (n === 0 || n === 1) return 1;  
  
    cache[n] = n * fatorial(n - 1);  
    return cache[n];  
  };  
}
```



Closures

vantagens

- protegem dados e detalhes da implementação (encapsulamento)
- mantém o estado sem depender de variáveis globais ou objetos externos

desvantagens

- dificuldade de depuração
- uso excessivo de memória
- comportamento inesperado em loops

exemplos

```
function criaContador() {  
  let contador = 0;  
  
  return function() {  
    contador++;  
    console.log(contador);  
  };  
}  
  
let contadorClique = criaContador();  
  
const botao = document.getElementById("botao");  
botao.addEventListener("click", contadorClique);
```

Currying

o que é

- transformação de uma função com vários parâmetros em uma sequência de funções com um parâmetro cada
- os argumentos podem ser "fornecidos aos poucos" os argumentos, criando funções cada vez mais específicas



Currying

vantagens

- cria funções especializadas a partir de uma função genérica
- economiza esforço em escrever novas funções

desvantagens

- pode tornar o código visualmente mais complexo e confuso
- perda de clareza

exemplos

```
// Função convencional
function multiplicar(a, b) {
  return a * b;
}
```

```
// Função com currying
function multiplicar(a) {
  return function(b) {
    return a * b;
  };
}
```

```
multiplicar(3)(10); // 30
```

```
const triplica = multiplicar(3);
triplica(10);
```



Currying

vantagens

- cria funções especializadas a partir de uma função genérica
- economiza esforço em escrever novas funções

desvantagens

- pode tornar o código visualmente mais complexo e confuso
- perda de clareza

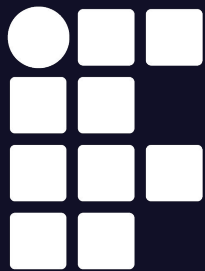
exemplos

```
function estilizarElemento(cor) {  
  return function (tamanho) {  
    return function (fonte) {  
      return function (elemento) {  
        elemento.style.color = cor;  
        elemento.style.fontSize = tamanho;  
        elemento.style.fontFamily = fonte;  
      };  
    };  
  };  
}
```

```
const estilizarTexto =  
  estilizarElemento('red')('20px')('Arial');
```

```
const elemento = document.getElementById('meuTexto');  
estilizarTexto(elemento);
```





**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript (parte 2)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas